

Introduction to Embedded Systems

Final Project

Implementation of an I^2C Serial EEPROM Driver

Team 13

Gulin Yildiz	s301348
Mihindukulasuriya Amandi Thisara Fernando	s321891
Mahipala Mudalige Kavinda Madushan Paris	s326875
Basthiyan Koralalage Jude Hirusha Rodrigo	s324843

June 2026

Contents

1 Introduction **2**
- 1.1 The I²C Protocol . . . 2
- 1.2 The M24M01 EEPROM . . . 2

2 Hardware Configuration **3**
- 2.1 Daughter Board Connection and Jumpers . . . 3
- 2.2 On-board Pull-up Resistors . . . 3
- 2.3 I²C3 Peripheral Configuration . . . 3
- 2.4 Device Address Derivation . . . 3
- 2.5 Memory Addressing . . . 4
- 2.6 Write Cycle Time . . . 4

3 The `memInit()` and `memWrite()` Functions **5**
- 3.1 `memInit()` . . . 5
  - 3.1.1 Purpose . . . 5
  - 3.1.2 Operation . . . 5
  - 3.1.3 Flowchart . . . 5
- 3.2 `memWrite()` . . . 7
  - 3.2.1 Purpose . . . 7
  - 3.2.2 Operation . . . 7
  - 3.2.3 Flowchart . . . 8

4 The `memRead()` Function and the `StartDefaultTask` Test Program **8**
- 4.1 `memRead()` . . . 8
  - 4.1.1 Purpose . . . 8
  - 4.1.2 Operation . . . 9
  - 4.1.3 Flowchart . . . 10
- 4.2 `StartDefaultTask` Test Program . . . 10
  - 4.2.1 Purpose . . . 10
  - 4.2.2 Operation . . . 10
  - 4.2.3 Flowchart . . . 12

5 Oscilloscope Results **13**
- 5.1 Write Transaction . . . 13
- 5.2 Write Cycle Delay (5 ms) . . . 15
- 5.3 Read Transaction . . . 15

6 Conclusion **19**

A Source Code **20**
- A.1 `mem.h` . . . 20
- A.2 `mem.c` . . . 20
- A.3 `application.c` . . . 22

1 Introduction

The goal of this laboratory experience is to implement a software **Application Programming Interface (API)** to access an I²C serial EEPROM from the VirtLAB board. The memory device used is the **M24M01-DFMN6TP**, a 128 KB (1 Mbit) I²C EEPROM mounted on the **X-NUCLEO-EEPRMA1** expansion (daughter) board. The firmware is built on top of the FreeRTOS template developed in the previous laboratories, and the new code provides three functions — `memInit`, `memWrite`, and `memRead` — which respectively verify the presence of the device, store a byte at a given address, and read a byte back. Correct operation of the API is verified in two complementary ways: through visual feedback on the on-board LEDs, and through direct observation of the I²C bus signals on a Digital Storage Oscilloscope.

The experiment makes use of the following equipment and components:

- VirtLAB board, based on the STM32L496 microcontroller (Cortex-M4, up to 80 MHz), acting as the I²C *master*.
- X-NUCLEO-EEPRMA1 daughter board carrying the M24M01-DFMN6TP EEPROM, acting as the I²C *slave*.
- A Digital Storage Oscilloscope (DSO) — Rigol DS1054Z — used to capture the read and write transactions on the SCL and SDA lines.
- PC running STM32CubeIDE, used to configure the peripherals, compile and flash the firmware.

1.1 The I²C Protocol

I²C (Inter-Integrated Circuit) is a synchronous, two-wire serial bus in which a single *master* controls communication with one or more *slave* devices. The two wires are **SCL** (the clock line, always driven by the master) and **SDA** (the bidirectional data line). Both lines are open-drain and held high by pull-up resistors, so any device may pull a line low but none may actively drive it high.

Every transaction begins with a **START** condition (SDA falling while SCL is high) and ends with a **STOP** condition (SDA rising while SCL is high). Between these, data travels one bit per clock pulse, grouped into 8-bit bytes, each followed by a 1-bit acknowledge (**ACK**) from the receiver. The master begins each transaction by transmitting the slave's device address; only the device matching that address responds.

1.2 The M24M01 EEPROM

The M24M01 is a non-volatile serial memory: it retains its contents when power is removed. It is organised as 131 072 byte-wide locations (2^{17} addresses), which is the reason a 17-bit memory addressing scheme is required, as discussed in Section 2.4. In this laboratory the device is accessed one byte at a time, both for writing and for reading. After each write the device requires a short internal programming interval before it will respond again, which the firmware accounts for with a fixed delay (see Section 2.6).

2 Hardware Configuration

2.1 Daughter Board Connection and Jumpers

The X-NUCLEO-EEPRMA1 daughter board is mounted on top of the VirtLAB board through the Arduino-compatible pin headers, which carry the I^2C signals, the supply, and ground between the two boards. The board has a single correct mating orientation. Before powering the system, the configuration jumpers on the VirtLAB board must be placed as indicated in Figure 1 of the project sheet, so that the Arduino connector pins are routed to the STM32 I^2C3 peripheral. Incorrect jumper placement prevents the I^2C signals from reaching the daughter board, in which case the device never acknowledges and `memInit` fails.

The I^2C bus signals are made accessible for measurement on a dedicated probe connector located on the top surface of the daughter board, as shown in Figure 2 of the project sheet. The oscilloscope probes for SCL and SDA are connected here, with their ground clips attached to a ground pin on the board.

2.2 On-board Pull-up Resistors

Because the I^2C lines are open-drain, both SCL and SDA require pull-up resistors to the supply rail. These are already present on the X-NUCLEO-EEPRMA1 daughter board (3.3 kΩ on each line), so no external resistors need to be added. Their value is appropriate for the 100 kHz bus speed used in this laboratory.

2.3 I^2C3 Peripheral Configuration

The I^2C3 peripheral of the STM32L496 was enabled and configured graphically in the STM32CubeMX `.ioc` editor. The relevant settings are:

- **Mode:** I^2C.
- **Pins:** `PC0` assigned to `I2C3_SCL` and `PC1` assigned to `I2C3_SDA`.
- **Speed:** 100 kHz (Standard mode), as required by the project specification.

After saving the `.ioc` file, the code is regenerated, which creates the peripheral handle `hi2c3` and the initialisation function `MX_I2C3_Init()` in `main.c`. The memory API in `mem.c` uses this handle (declared with `extern`) for all bus transactions.

2.4 Device Address Derivation

To address the EEPROM on the bus, the master sends an 8-bit *device select code*. According to Table 2 of the M24M01 datasheet, this byte has the following structure:

Table 1: Device select code (control byte) structure of the M24M01.

b7	b6	b5	b4	b3	b2	b1	b0
1	0	1	0	E2	E1	A16	R/$\overline{\text{W}}$

The individual fields are determined as follows:

- The four most significant bits `1010` are a fixed device-type identifier, common to all ST serial EEPROMs.
- **E2** and **E1** are set by the corresponding hardware chip-enable pins. On the X-NUCLEO-EEPRMA1 board these are tied to V_{CC}, so both are read as logic `1`.
- **A16** is the most significant bit (bit 16) of the 17-bit memory address. Since all test addresses used in this laboratory lie below `0x10000` (the lower 64 KB half of the device), A16 = 0.
- **R/$\overline{\mathbf{W}}$** selects the direction: `0` for a write, `1` for a read. This bit is handled automatically by the HAL functions, so the firmware always passes the write form of the address.

Combining these fields for a write operation gives:

$$\underbrace{1010}_{\text{fixed}}\ \underbrace{1}_{\text{E2}}\ \underbrace{1}_{\text{E1}}\ \underbrace{0}_{\text{A16}}\ \underbrace{0}_{\text{R}/\overline{\text{W}}} = 1010\,1100_2 = \mathbf{0xAC} \tag{1}$$

The constant `0xAC` is therefore used as the device address throughout the firmware. For a read the HAL layer internally sets bit 0, producing `0xAD`.

2.5 Memory Addressing

The M24M01 contains $2^{17} = 131\,072$ byte locations, which cannot be addressed by a single 8-bit byte (maximum 256 locations). The addressing is therefore split: the most significant bit (A16) is carried inside the device select code (Table 1), while the remaining 16 bits are sent as two separate address bytes (high byte followed by low byte) after the device address. In the firmware this is requested by passing the `I2C_MEMADD_SIZE_16BIT` parameter to the HAL memory functions, which instructs the HAL to transmit the 16-bit memory address as two bytes.

2.6 Write Cycle Time

After a write transaction completes on the bus, the EEPROM begins an internal programming cycle during which it physically stores the received byte into its memory cells. According to Table 14 of the M24M01 datasheet, the maximum internal write cycle duration is $t_W = 5\,\text{ms}$. While this cycle is in progress the device does not acknowledge any further requests on the bus — it responds with a NACK to every transaction.

For this reason, the `memWrite` function performs a fixed delay of 5 ms (`HAL_Delay(5)`) immediately after each write. Omitting this delay would cause a subsequent read or write to fail, because the device would still be busy programming and would not acknowledge. This delay is the single most important timing constraint of the memory API.

3 The `memInit()` and `memWrite()` Functions

3.1 `memInit()`

3.1.1 Purpose

The `memInit()` function verifies that the M24M01-DFMN6TP EEPROM is physically present on the I2C bus and responding correctly before any read or write operations are attempted. This initialization step prevents the application from communicating with a missing device or a bus that is not functioning correctly.

3.1.2 Operation

The function uses the STM32 HAL function:

```
HAL_I2C_IsDeviceReady(&hi2c3, MEM_DEV_ADDR, 3, 100);
```

where `MEM_DEV_ADDR` is defined as `0xAC`. The function transmits the EEPROM device address on the I2C bus and waits for an acknowledgement (ACK) from the EEPROM.

The parameters specify:

- `&hi2c3` – handle for the I2C3 peripheral.
- `0xAC` – EEPROM device address.
- `3` – number of retry attempts.
- `100` – timeout value in milliseconds for each attempt.

The retry mechanism is useful because the EEPROM may be momentarily busy (for example, still completing an internal write cycle from a previous operation) and may not acknowledge immediately. By attempting the address check up to 3 times, transient busy states are tolerated without falsely reporting the device as absent.

On the I2C bus, the transaction generated is:

$$\text{START} \rightarrow \text{0xAC} \rightarrow \text{ACK?} \rightarrow \text{STOP}$$

If the EEPROM acknowledges the address by pulling SDA low during the ACK clock pulse, the device is considered present and the function returns `0`.

If no acknowledgement is received after all retry attempts, the function returns `-1`, indicating either a missing EEPROM or an I2C communication error.

In the application code, a return value of `-1` causes LED0 to blink rapidly and the program halts, indicating an initialization failure.

3.1.3 Flowchart

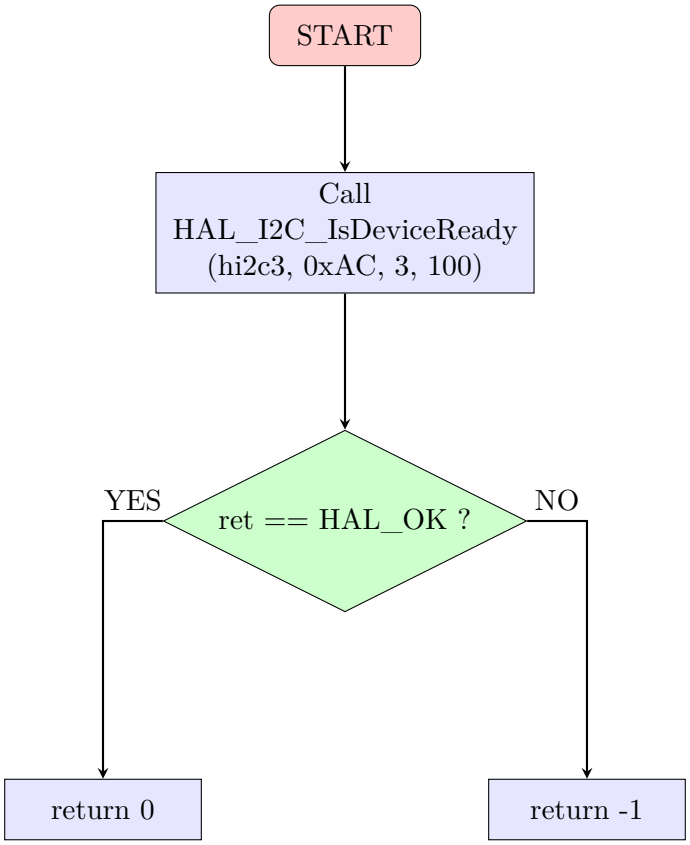


Figure 1: Flowchart of memInit()

3.2 `memWrite()`

3.2.1 Purpose

The `memWrite()` function writes a single byte of data to a specified 16-bit memory address within the M24M01 EEPROM.

3.2.2 Operation

The function uses the STM32 HAL memory-write function:

```
HAL_I2C_Mem_Write(
    &hi2c3,
    MEM_DEV_ADDR,
    address,
    I2C_MEMADD_SIZE_16BIT,
    &data,
    1,
    HAL_MAX_DELAY
);
```

The parameters specify:

- `&hi2c3` – I2C3 peripheral handle.
- `0xAC` – EEPROM device address.
- `address` – target EEPROM memory location.
- `I2C_MEMADD_SIZE_16BIT` – memory address is transmitted as two bytes.
- `&data` – pointer to the byte being written.
- `1` – write one byte.
- `HAL_MAX_DELAY` – block until the transfer completes.

The `I2C_MEMADD_SIZE_16BIT` parameter is required because the EEPROM contains more than 256 memory locations and therefore requires a 16-bit address to access its full range – a single byte cannot represent all possible addresses. The HAL library automatically generates the complete I2C write transaction:

$$\text{START} \rightarrow \text{0xAC} \rightarrow \text{ACK} \rightarrow \text{Address High Byte} \rightarrow \text{ACK} \rightarrow \text{Address Low Byte} \rightarrow \text{ACK} \rightarrow \text{Data Byte} \rightarrow \text{ACK} \rightarrow \text{STOP}$$

After the write transaction completes successfully, the EEPROM begins its internal programming cycle. According to the M24M01 datasheet, the maximum write-cycle time (t_W) is 5 ms. During this period, the EEPROM does not respond to I2C requests and will NACK any attempted communication, since it is physically burning the data into its memory cells. Therefore, the code executes:

```
HAL_Delay(5);
```

to ensure the write operation has fully completed before any subsequent access occurs.

If `HAL_I2C_Mem_Write()` returns `HAL_OK`, the function waits 5 ms via `HAL_Delay(5)` and then returns `0`. If an I2C error occurs, the function immediately returns `-1` without

waiting.

3.2.3 Flowchart

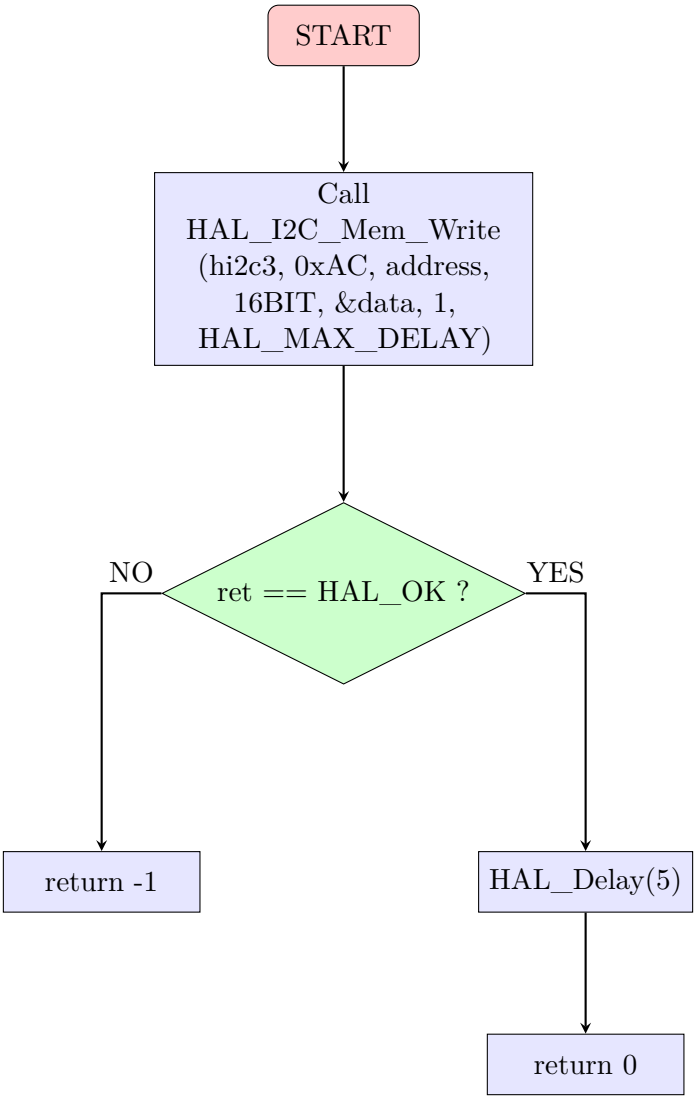


Figure 2: Flowchart of memWrite()

4 The `memRead()` Function and the `StartDefaultTask` Test Program

4.1 `memRead()`

4.1.1 Purpose

The `memRead()` function reads a single byte from a specified 16-bit memory address within the M24M01 EEPROM and places the received value into the caller's variable via a pointer. Before any data can be read, the function must instruct the EEPROM which memory location to return; this requires a more complex bus sequence than a simple write, as described below.

4.1.2 Operation

The function uses the STM32 HAL memory-read function:

```
HAL_I2C_Mem_Read(
    &hi2c3,
    MEM_DEV_ADDR,
    address,
    I2C_MEMADD_SIZE_16BIT,
    dataPtr,
    1,
    HAL_MAX_DELAY
);
```

The parameters specify:

- `&hi2c3` – I2C3 peripheral handle.
- `0xAC` – EEPROM device address (write form; the HAL sets the R/W bit to 1 internally for the read phase).
- `address` – target EEPROM memory location.
- `I2C_MEMADD_SIZE_16BIT` – memory address is transmitted as two bytes, as required by the 17-bit address space of the M24M01.
- `dataPtr` – pointer to the variable where the received byte is stored directly in the caller's memory.
- `1` – read exactly one byte.
- `HAL_MAX_DELAY` – block until the transfer completes.

A read from a random address requires a two-phase transaction. In the first phase (the *dummy write*), the master sends the device address in write mode followed by the two memory-address bytes, exactly as it would for a write. This loads the EEPROM's internal address counter with the target location. In the second phase, the master issues a **Repeated START** condition — a new START without releasing the bus with a STOP — and re-sends the device address, this time in read mode (`0xAD`). The EEPROM then drives the SDA line and outputs the byte stored at the previously loaded address.

The complete bus sequence generated automatically by `HAL_I2C_Mem_Read()` is:

$$\texttt{START} \rightarrow \texttt{0xAC} \rightarrow \text{ACK} \rightarrow \text{Addr Hi} \rightarrow \text{ACK} \rightarrow \text{Addr Lo} \rightarrow \text{ACK}$$
$$\rightarrow \textbf{Repeated START} \rightarrow \texttt{0xAD} \rightarrow \text{ACK} \rightarrow \text{Data (EEPROM drives SDA)} \rightarrow \text{NACK} \rightarrow \text{STOP}$$

The master sends a NACK on the last byte to signal that no further bytes are required, after which a STOP condition is generated. Releasing the bus between the two phases (i.e. issuing a STOP before the second START) would cause the EEPROM to discard the loaded address, making the Repeated START mandatory.

Unlike `memWrite()`, no delay is needed after a read: the EEPROM does not enter an internal programming cycle when it sends data, so it is immediately ready for the next transaction.

If `HAL_I2C_Mem_Read()` returns `HAL_OK`, the function returns 0. If an I2C error occurs, the function returns −1.

4.1.3 Flowchart

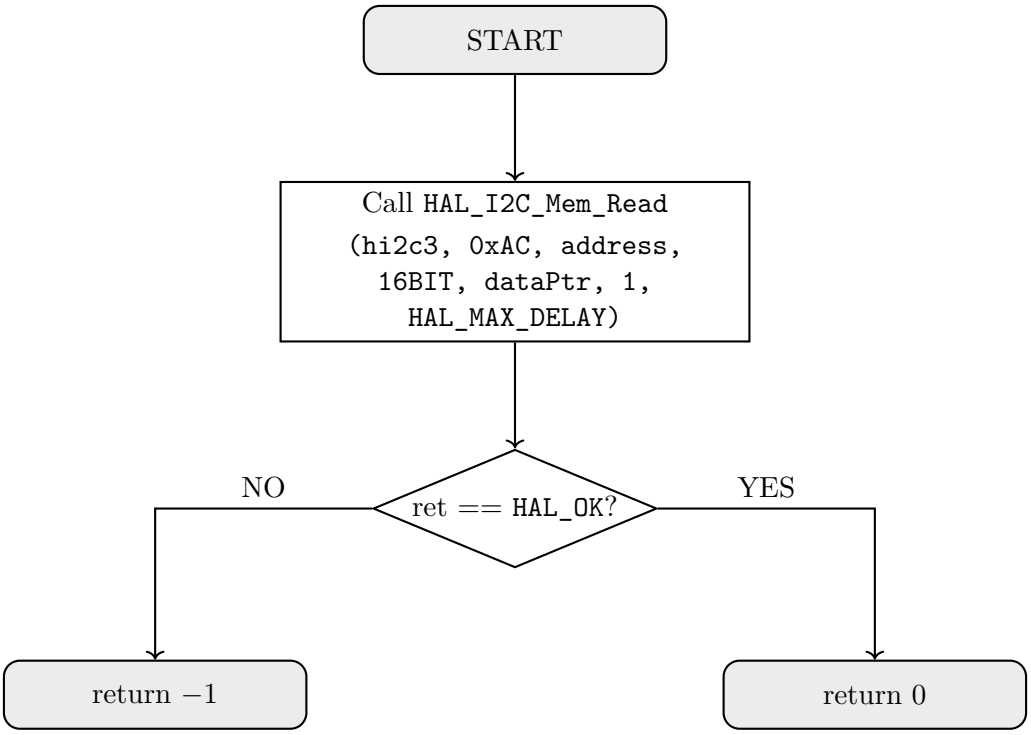


Figure 3: Flowchart of `memRead()`

4.2 `StartDefaultTask` Test Program

4.2.1 Purpose

The `StartDefaultTask` function serves as the top-level test harness for the EEPROM API. It exercises all three API functions in sequence — `memInit`, `memWrite`, and `memRead` — verifies that written data is correctly read back, reports the overall result through the on-board LEDs, and then enters a continuous loop that keeps the I2C bus active so that the transactions can be observed with the oscilloscope.

4.2.2 Operation

After creating the event flags object required by the acquisition task, the function immediately proceeds with the EEPROM test. The test consists of four steps.

Step 1 — Initialisation. `memInit()` is called to confirm that the EEPROM is present and acknowledging on the I2C bus. If it returns −1, LED0 blinks rapidly at 100 ms intervals in an infinite loop, halting all further execution and signalling a hardware or wiring fault.

Step 2 — Five writes. A `for` loop calls `memWrite()` once for each of five address/data pairs. The addresses are spread across different regions of the memory space: `0x0010`, `0x00FF`, `0x0100`, `0x1000`, and `0x1234`. The corresponding data bytes are `0x42`, `0xA5`,

`0x7E`, `0x5A`, and `0x3C`; all values differ from `0x00` and `0xFF` so that a genuine write can be distinguished from a blank or stuck memory cell. Any failed write increments the `errors` counter.

Step 3 — Five reads and comparison. A second `for` loop calls `memRead()` for each address, storing the result in `readBack`. Both a failed read (return value -1) and a data mismatch (`readBack != testData[i]`) increment the `errors` counter, so the counter captures every type of failure.

Step 4 — Result and continuous loop. If `errors` equals zero, all four LEDs blink together at 1 Hz (500 ms on, 500 ms off), confirming that all five write/read pairs passed. At the start of each iteration the task also performs one `memWrite` followed by one `memRead` on address `0x0010`, keeping the I2C bus continuously active for oscilloscope observation. If `errors` is non-zero, LED0 stays solid and the task blocks indefinitely, indicating a test failure.

4.2.3 Flowchart

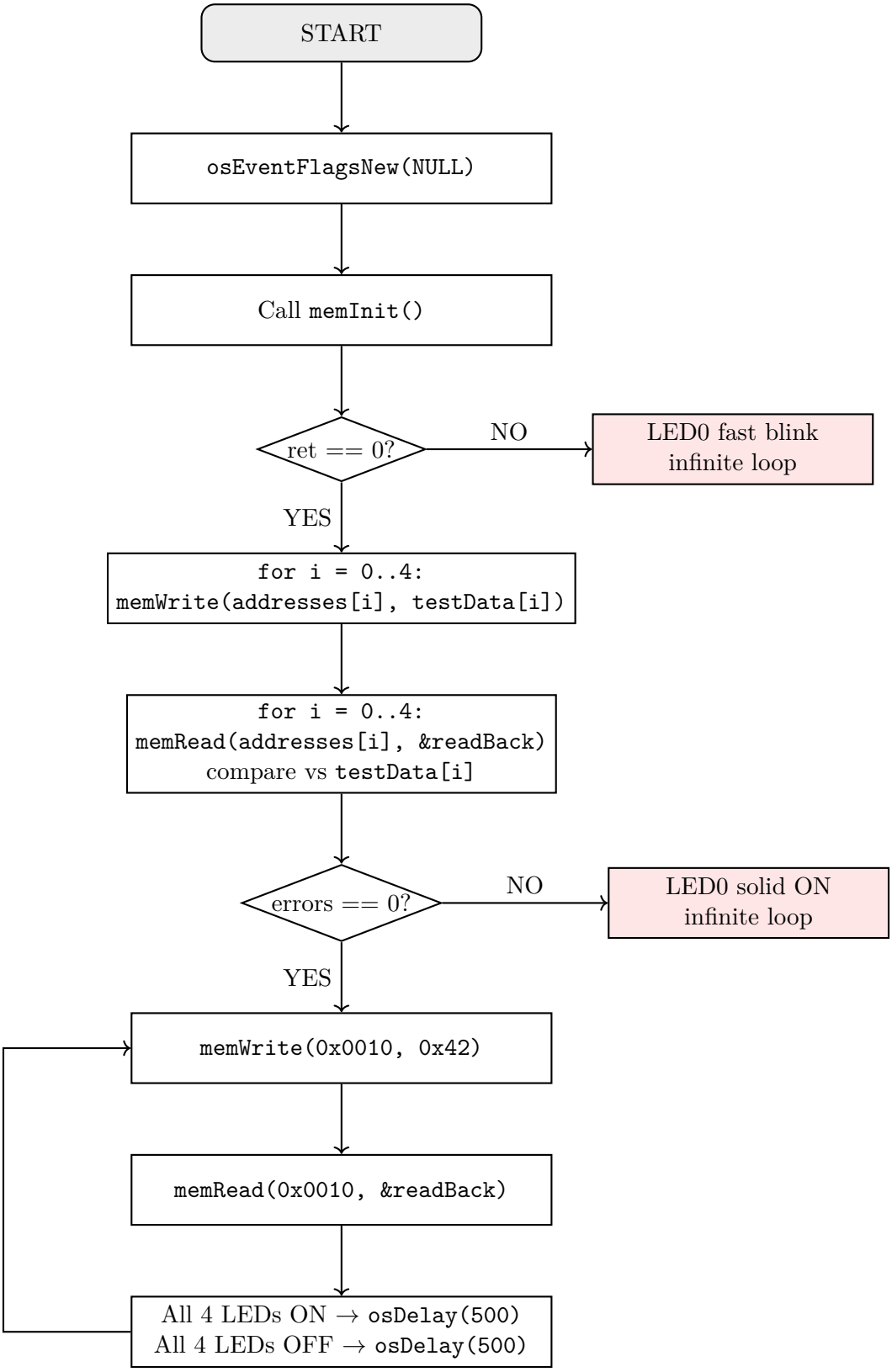


Figure 4: Flowchart of `StartDefaultTask`

5 Oscilloscope Results

All measurements were performed using the Rigol DS1054Z oscilloscope. Channel 1 (CH1, yellow) was connected to the SDA line and Channel 3 (CH3, purple) was connected to the SCL line of the I²C bus. The firmware was configured to continuously repeat one write followed by one read on address `0x0010` inside `StartDefaultTask`, keeping the bus active for observation. The trigger was set to a falling edge on CH1 (SDA) in Normal sweep mode, with a threshold of 1.48 V.

5.1 Write Transaction

A byte write to address `0x0010` with data value `0x42` was captured at a horizontal timebase of 10 µs/div. Because the full transaction spans approximately 400 µs at 70.4 kHz, the capture was panned across the transaction in sequential frames. Two representative frames are presented: one showing the START condition at the beginning of the transaction, and one showing the STOP condition and bus release at the end.

Start of Write Transaction



Figure 5: Write transaction — START condition and beginning of device address byte 0xAC. CH1 (yellow) = SDA, CH3 (purple) = SCL. Timebase: 10 µs/div, D = 45.6 µs, Freq = 70.4 kHz, Vpp = 3.28 V.

Figure 5 shows the beginning of the write transaction. On the SDA trace (yellow), the line drops low while SCL remains high — this is the **START condition** as defined by the I²C protocol. Following the START, the master transmits the device select byte `0xAC` (binary `1010 1100`), which encodes the device type identifier (`1010`), the chip enable bits E2 = 1 and E1 = 1 (both pins tied to V_{CC} on the X-NUCLEO-EEPRMA1 board), the most significant address bit A16 = 0 (all test addresses are below `0x10000`), and the R/$\bar{W}$ bit set to 0 indicating a write operation. The EEPROM acknowledges by pulling SDA low during the 9th clock pulse (ACK). The measured SCL frequency of 70.4 kHz is close to the configured 100 kHz; the small deviation is normal and results from I²C clock stretching and bus capacitance effects introduced by the pull-up resistors R5 and R6 (3.3 kΩ each)

on the X-NUCLEO-EEPRMA1 board. The signal swing of 3.28 V is consistent with the 3.3 V supply rail.

By manually reading the SDA line during each SCL high phase, the first eight bits after the START condition correspond to high–low–high–low–high–high–low–low = binary `1010 1100` = `0xAC`, confirming the correct device address and $\mathrm{R}/\overline{\mathrm{W}} = 0$ for a write operation.

End of Write Transaction and Bus Release



Figure 6: Write transaction — STOP condition and bus release into the internal write cycle. CH1 (yellow) = SDA, CH3 (purple) = SCL. Timebase: 10 µs/div, D = 403.6 µs, Freq = 70.4 kHz, Vpp = 3.28 V.

Figure 6 captures the end of the write transaction. After the data byte (`0x42`) has been transmitted and acknowledged, the master generates a **STOP condition**: SDA rises while SCL is stable and high. Both lines then remain in the idle high state. At this point the EEPROM begins its internal write cycle (t_W), during which it physically programs the data into the memory cell array. According to Table 14 of the M24M01-R datasheet, the maximum write cycle duration is $t_W = 5$ ms. During this window the EEPROM will NACK any new I^{2}C transaction, which is why `HAL_Delay(5)` is called immediately after every `memWrite()` operation.

5.2 Write Cycle Delay (5 ms)



Figure 7: Write-to-read gap confirming the mandatory 5 ms EEPROM internal write cycle. CH1 (yellow) = SDA, CH3 (purple) = SCL. Timebase: 1.00 ms/div, D = 2.460 ms, Vpp = 3.44 V.

Figure 7 was captured at a timebase of 1.00 ms/div, giving a total window of 10 ms — wide enough to show both the write and read transactions together with the full gap between them. The left burst of I^2C activity on SDA and SCL is the tail end of the write transaction. The right burst is the beginning of the subsequent read transaction. Between the two bursts, both SDA and SCL remain continuously high, confirming the bus is fully idle. This silent period corresponds to the `HAL_Delay(5)` call in `memWrite()`, which enforces the mandatory 5 ms write cycle wait required by the M24M01-R before any new operation can be initiated. Without this delay the EEPROM would still be programming its memory cells and would NACK the following read, causing a bus error.

5.3 Read Transaction

A byte read from address `0x0010` was captured at a horizontal timebase of 10 µs/div. The read transaction consists of two separate bursts separated by a Repeated START condition. The first burst (dummy write phase) loads the target memory address into the EEPROM's internal address counter without releasing the bus. The second burst (read phase) retrieves the data byte stored at that address. This two-burst structure is generated automatically by `HAL_I2C_Mem_Read()` when called with the `I2C_MEMADD_SIZE_16BIT` parameter.

Start of Read Transaction — Dummy Write Phase



Figure 8: Read transaction — START condition and beginning of dummy write phase. CH1 (yellow) = SDA, CH3 (purple) = SCL. Timebase: 10 µs/div, D = 6.042 ms, Freq = 70.4 kHz, Vpp = 3.28 V.

Figure 8 shows the opening of the read transaction. SDA drops low while SCL remains high, generating a **START condition**. The master then transmits the device select byte `0xAC` with $\mathrm{R}/\bar{\mathrm{W}} = 0$ (write mode), followed by the 16-bit memory address split across two bytes: address high byte `0x00` and address low byte `0x10`. This is the **dummy write phase** — the master behaves as if performing a write but sends no data byte. Its sole purpose is to load address `0x0010` into the EEPROM's internal address counter so that the subsequent read retrieves data from the correct location. The `I2C_MEMADD_SIZE_16BIT` flag in `HAL_I2C_Mem_Read()` instructs HAL to send the address as two bytes, which is required because the M24M01-R has more than 256 addressable locations.

By manually reading the SDA line during each SCL high phase in Figure 8, the first eight bits transmitted after the START condition are high–low–high–low–high–high–low–low, corresponding to binary `1010 1100` = `0xAC`, confirming $\mathrm{R}/\bar{\mathrm{W}} = 0$ (write mode) during the dummy write phase. The device select byte switches to `0xAD` (binary `1010 1101`, $\mathrm{R}/\bar{\mathrm{W}} = 1$) only after the Repeated START condition visible in Figure 9, at which point the master switches to read mode and the EEPROM begins driving SDA to return the requested data byte.

Repeated START Condition



Figure 9: Read transaction — Repeated START (Sr) condition transitioning from dummy write phase to read phase. CH1 (yellow) = SDA, CH3 (purple) = SCL. Timebase: 10 µs/div, D = 6.186 ms, Vpp = 3.28 V.

Figure 9 is the most important capture of the read transaction. At the centre of the frame, after the final byte of the dummy write phase, SDA rises while SCL is high — resembling a STOP condition — but then immediately falls again while SCL remains high. This sequence is the **Repeated START (Sr)** condition. No STOP is issued between the two bursts; releasing the bus would cause the EEPROM to discard the address that was just loaded into its internal counter. By issuing Sr instead, the master retains ownership of the bus and switches to read mode without losing the address pointer. Immediately after Sr, the master transmits the device select byte `0xAD` (`0xAC` with $R/\bar{W} = 1$), to which the EEPROM responds with an ACK and begins driving SDA to output the requested data byte.

The “Real” Read Transaction



Figure 10: Read transaction — device select byte `0xAD` transmitted after Repeated START, confirming switch to read mode. CH1 (yellow) = SDA, CH3 (purple) = SCL. Timebase: 10 µs/div, D = 6.307 ms, Freq = 62.5 kHz, Vpp = 3.36 V.

Figure 10 shows the device select byte transmitted immediately after the Repeated START condition of Figure 9. By manually reading the SDA line during each SCL high phase, the eight bits are high–low–high–low–high–high–low–high = binary `1010 1101` = `0xAD`, confirming that the R/$\bar{W}$ bit has switched from 0 to 1. This single bit change — from `0xAC` to `0xAD` — marks the transition from the dummy write phase to the active read phase. The EEPROM responds with an ACK on the 9th clock pulse and then takes over driving SDA to return the data byte stored at address `0x0010`. This frame therefore confirms both the correct device addressing and the correct protocol switch into read mode.

End of Read Transaction — NACK and STOP



Figure 11: Read transaction — master NACK and STOP condition, bus released after data byte received. CH1 (yellow) = SDA, CH3 (purple) = SCL. Timebase: 10 µs/div, D = 6.473 ms, Freq = 33.3 kHz, Vpp = 3.36 V.

Figure 11 shows the conclusion of the read transaction. After the EEPROM drives SDA to output the data byte, the master deliberately does **not** acknowledge the received byte — it issues a **NACK** (SDA remains high during the 9th clock pulse) to signal that no further bytes are required. The master then generates a **STOP condition** (SDA rises while SCL is stable and high), releasing the bus. Both SDA and SCL return to the idle high state, confirming the transaction is complete. No `HAL_Delay()` is required after a read operation because the EEPROM does not enter a busy write cycle after sending data.

6 Conclusion

This project successfully demonstrated the implementation of a three-function software API (`memInit`, `memWrite`, `memRead`) for interfacing an STM32L496 microcontroller with an ST M24M01-R 1 Mbit I^{2}C EEPROM over the I^{2}C3 peripheral using the STM32 HAL library. The `StartDefaultTask` test program executed five write and five read operations across different memory addresses spanning the full 17-bit address space of the device, and all five write/read pairs produced matching data values, confirmed by all four LEDs blinking at 1 Hz upon completion. Oscilloscope measurements on the SDA and SCL lines verified correct I^{2}C bus operation at approximately 70.4 kHz with a signal swing of approximately 3.28–3.44 V, consistent with the 3.3 V supply. The write transaction exhibited the expected four-byte structure — device address byte, memory address high byte, memory address low byte, and data byte — each followed by an ACK from the EEPROM. The read transaction displayed the correct two-burst Repeated START structure, with the dummy write phase setting the target address and the subsequent read phase retrieving the stored data byte, terminated by a master NACK and STOP condition. The mandatory 5 ms internal write cycle delay was confirmed by the clearly visible idle period on the bus between the write

and read transactions, consistent with the t_W specification in Table 14 of the M24M01-R datasheet.

A Source Code

This appendix contains the complete, commented source code of the three files implementing and testing the EEPROM driver: the API header `mem.h`, its implementation `mem.c`, and the test program in `application.c`.

A.1 `mem.h`

```c
/*
 * mem.h
 *
 * API for accessing the M24M01-DFMN6TP serial EEPROM
 * over the I2C3 peripheral at 100kHz.
 */

#ifndef INC_MEM_H_
#define INC_MEM_H_

#include <stdint.h>

// Initialize the serial memory subsystem.
// Verifies the EEPROM is reachable on the I2C bus.
// Returns 0 on success, -1 if the device does not respond.
int memInit(void);

// Read a single byte from the specified memory address.
// The result is written into the location pointed to by dataPtr.
// Returns 0 on success, -1 on I2C error.
int memRead(uint16_t address, uint8_t *dataPtr);

// Write a single byte (data) to the specified memory address.
// Waits for the EEPROM internal write cycle to complete before
   returning.
// Returns 0 on success, -1 on I2C error.
int memWrite(uint16_t address, uint8_t data);

#endif /* INC_MEM_H_ */
```

Listing 1: mem.h – EEPROM API declarations

A.2 `mem.c`

```c
/*
 * mem.c
 *  Created on: Jun 3, 2026
 *      Author: jamiu
 *
 * Implementation of the serial memory API for the M24M01-DFMN6TP EEPROM
   .
 * Communication uses the I2C3 peripheral configured at 100kHz.
 *
 * Device address structure (control byte):
```

```
 *   Bits [7:4] = 1010  -- fixed code for all serial EEPROMs
 *   Bit  [3]   = E2    -- hardware pin, tied to VCC on X-NUCLEO-EEPRMA1 = 1
 *   Bit  [2]   = E1    -- hardware pin, tied to VCC on X-NUCLEO-EEPRMA1 = 1
 *   Bit  [1]   = A16   -- MSB of memory address (bit 16), 0 for lower 64KB
 *   Bit  [0]   = R/W   -- 0 = write, 1 = read (set internally by HAL)
 *
 *   With E2=1, E1=1, A16=0, R/W=0: 1010 1100 = 0xAC
 */

#include "mem.h"
#include "main.h"

// I2C3 handle generated by CubeMX in main.c
extern I2C_HandleTypeDef hi2c3;

// Write address for the EEPROM (E2=1, E1=1, A16=0, R/W=0)
// HAL will automatically set the R/W bit to 1 for read operations
#define MEM_DEV_ADDR  0xACU

// Maximum write cycle time per M24M01 datasheet (tW = 5ms)
// The EEPROM NACKs all requests during this period
#define MEM_WRITE_CYCLE_MS  5U


// memInit -- ping the EEPROM to confirm it is present and responsive
int memInit(void) {
    HAL_StatusTypeDef ret;

    // Send the device address and check for ACK
    // 3 retries, 100ms timeout per attempt
    ret = HAL_I2C_IsDeviceReady(&hi2c3, MEM_DEV_ADDR, 3, 100);

    if (ret != HAL_OK) {
        return -1;  // no ACK received -- device not found or bus error
    }
    return 0;
}


// memWrite -- write a single byte to the specified 16-bit memory address
// HAL generates: START -> [0xAC] -> ACK -> [addr_hi] -> ACK -> [addr_lo] -> ACK -> [data] -> ACK -> STOP
int memWrite(uint16_t address, uint8_t data) {
    HAL_StatusTypeDef ret;

    ret = HAL_I2C_Mem_Write(
        &hi2c3,                 // I2C3 peripheral handle
        MEM_DEV_ADDR,           // EEPROM device address (0xAC)
        address,                // target memory address (16-bit word address)
        I2C_MEMADD_SIZE_16BIT,  // address sent as two bytes: high byte then low byte
        &data,                  // pointer to the byte to be written
        1,                      // write exactly one byte
```

```
61          HAL_MAX_DELAY           // block until transfer completes
62      );
63
64      if (ret != HAL_OK) {
65          return -1;
66      }
67
68      // Wait for the internal EEPROM write cycle to complete
69      // The chip NACKs all I2C access during this window
70      HAL_Delay(MEM_WRITE_CYCLE_MS);
71
72      return 0;
73  }
74
75
76  // memRead -- read a single byte from the specified 16-bit memory
        address
77  // HAL generates: START -> [0xAC] -> ACK -> [addr_hi] -> ACK -> [addr_lo
        ] -> ACK
78  //                -> Repeated START -> [0xAD] -> ACK -> [data from
        EEPROM] -> NACK -> STOP
79  int memRead(uint16_t address, uint8_t *dataPtr) {
80      HAL_StatusTypeDef ret;
81
82      ret = HAL_I2C_Mem_Read(
83          &hi2c3,                 // I2C3 peripheral handle
84          MEM_DEV_ADDR,           // EEPROM device address (HAL sets R/W=1
        internally)
85          address,                // target memory address (16-bit word
        address)
86          I2C_MEMADD_SIZE_16BIT,  // address sent as two bytes: high byte
        then low byte
87          dataPtr,                // pointer to buffer where received byte
        is stored
88          1,                      // read exactly one byte
89          HAL_MAX_DELAY           // block until transfer completes
90      );
91
92      if (ret != HAL_OK) {
93          return -1;
94      }
95
96      return 0;
97  }
```

Listing 2: mem.c – EEPROM API implementation

A.3 application.c

```
1  /*
2   * application.c
3   *
4   *  Created on: Oct 20, 2020
5   *      Author: max
6   */
7
8  #include "main.h"
```

```c
#include "cmsis_os2.h"
#include "io.h"
#include "mem.h"


// Event flags object used to signal the acquisition task from the timer callback
osEventFlagsId_t acqEventHandle;


void StartDefaultTask( void *argument ) {

    // Create the event flag object so the acquisition task can block safely
    acqEventHandle = osEventFlagsNew(NULL);


    // EEPROM API Test

    // Test addresses -- spread across the memory space, none are zero
    const uint16_t addresses[] = { 0x0010, 0x00FF, 0x0100, 0x1000, 0x1234 };
    // Test data -- all values differ from 0x00 and 0xFF as required
    const uint8_t  testData[]  = { 0x42,   0xA5,   0x7E,   0x5A,   0x3C };
    const int      N_TESTS     = 5;
    uint8_t        readBack;
    int            errors      = 0;

    // Step 1: Verify the EEPROM is reachable on the I2C bus
    if (memInit() != 0) {
        // Rapid LED0 blink signals that the EEPROM was not found
        while (1) {
            led0(ON);  osDelay(100);
            led0(OFF); osDelay(100);
        }
    }

    // Step 2: Write each test byte to its corresponding address
    for (int i = 0; i < N_TESTS; i++) {
        if (memWrite(addresses[i], testData[i]) != 0) {
            errors++;
        }
    }

    // Step 3: Read back each address and compare against the written value
    for (int i = 0; i < N_TESTS; i++) {
        if (memRead(addresses[i], &readBack) != 0) {
            errors++;                       // read operation failed
        } else if (readBack != testData[i]) {
            errors++;                       // data mismatch -- write or read error
        }
    }

    // Step 4: Signal overall result using LEDs
    if (errors == 0) {
```

```
61          // All tests passed -- keep writing and reading so oscilloscope
    can capture
62          while (1) {
63              memWrite(0x0010, 0x42);
64              memRead(0x0010, &readBack);
65
66              // Blink LEDs to confirm pass
67              led0(ON); led1(ON); led2(ON); led3(ON);
68              osDelay(500);
69              led0(OFF); led1(OFF); led2(OFF); led3(OFF);
70              osDelay(500);
71          }
72      } else {
73          // One or more tests failed -- LED0 stays solid
74          led0(ON);
75          while (1) { osDelay(1000); }
76      }
77  }
```

Listing 3: application.c – StartDefaultTask test program